
Playwright no Teste Funcional

Do caso de teste à evidência automatizada



Prof. Dr. João Choma

Teste e Qualidade de Software

github.com/JoaoChoma

Semana 05



Automatizando jornadas reais em aplicações web com TypeScript.



Ao final, você deverá ser capaz de:

- explicar o papel do Playwright no teste funcional;
- descrever como navegador, contexto, página e locator se relacionam;
- instalar e configurar um projeto;
- implementar um teste funcional completo;
- selecionar locators e asserções resistentes;
- executar, depurar e interpretar relatórios;
- reconhecer o que deve — e o que não deve — ser automatizado.



Na aula anterior, projetamos casos a partir de:

- requisitos e oráculos;
- classes de equivalência;
- valores-limite;
- estados e sequências;
- efeitos observáveis.

O Playwright **não decide o que testar**. Ele executa no navegador o raciocínio que já foi documentado no caso de teste.



Regra: uma pessoa com credenciais válidas e conta ativa deve acessar sua área. Credenciais inválidas devem ser rejeitadas sem revelar qual dado está incorreto.

- Caso — Entrada — Resultado esperado
- login válido — e-mail e senha corretos — área autenticada visível
- senha inválida — e-mail válido e senha errada — acesso negado
- e-mail inválido — formato incorreto — validação ou rejeição
- conta bloqueada — credenciais corretas — acesso negado com orientação

O teste automatizado precisa reproduzir condição, ação e verificação.



Playwright é uma plataforma de automação para aplicações web. Com o **Playwright Test**, temos no mesmo conjunto:

- executor de testes;
- asserções;
- isolamento;
- paralelismo;
- configuração de navegadores e dispositivos;
- relatórios e rastros de execução;
- ferramentas para gerar e depurar testes.

Ele suporta Chromium, Firefox e WebKit, localmente ou em integração contínua.



```
Regra de negócio
  ↓
Caso de teste funcional
  ↓
Código Playwright
  ↓
Navegador + aplicação + dependências
  ↓
Asserções + relatório + trace
```

Ideia central: o Playwright automatiza a interação e a observação; o oráculo ainda vem da regra de negócio.



Em parte.

Ele interage com elementos reais da página:

- navega para uma URL;
- preenche campos;
- seleciona opções;
- envia formulários;
- abre páginas e pop-ups;
- observa texto, estado, URL e requisições.

Mas a execução é programática e pode usar recursos que uma pessoa não possui, como interceptar rede ou reutilizar estado de autenticação.



Playwright Test

```
|-- worker 1 -> browser -> context -> page  
|-- worker 2 -> browser -> context -> page  
`-- worker 3 -> browser -> context -> page
```

- **browser:** processo do Chromium, Firefox ou WebKit;
- **context:** sessão isolada, semelhante a um perfil anônimo;
- **page:** aba ou janela;
- **locator:** forma de reencontrar um elemento;
- **fixture:** recurso preparado para o teste, como page.



Cada teste deve começar em um estado conhecido.

O contexto do navegador isola:

- cookies;
- armazenamento local;
- cache e permissões;
- páginas abertas;
- estado de autenticação.

Dica: testes independentes podem rodar em qualquer ordem e em paralelo. Se um teste depende do anterior, a suíte está escondendo uma pré-condição.



Um locator descreve como encontrar um elemento no momento da ação.

```
page.getByRole('button', { name: 'Entrar' })  
page.getByLabel('E-mail')  
page.getByPlaceholder('nome@exemplo.com')  
page.getByText('Pedido confirmado')  
page.getByTestId('saldo-disponivel')
```

Prefira atributos percebidos pelo usuário: papel, nome acessível, rótulo e texto.



```
await page.getByRole('button', { name: 'Entrar' }).click();
```

Esse locator expressa:

- o tipo de controle esperado;
- o nome que o usuário percebe;
- uma relação próxima da árvore de acessibilidade.

Sinal útil: se o botão não possui nome acessível, talvez exista também um problema de acessibilidade no produto.



```
// Frágil: depende da estrutura e das classes de CSS
page.locator('div.login > form > button.btn-primary')

// Melhor: descreve a função do elemento
page.getByRole('button', { name: 'Entrar' })
```

Classes, IDs gerados e posições no DOM mudam com refatorações visuais sem alterar o comportamento.

Use `data-testid` quando não houver um identificador semântico estável.



Antes de executar uma ação, o Playwright verifica condições como:

- locator resolvido para o elemento esperado;
- elemento visível;
- elemento estável;
- elemento habilitado;
- possibilidade real de receber a ação.

Isso evita muitos `sleep` e temporizadores fixos.

Atenção: auto-wait não corrige um oráculo ruim nem espera por qualquer condição de negócio imaginada pelo teste.



```
await expect(page.getByRole('heading', {  
  name: 'Minha conta'  
})).toBeVisible();  
  
await expect(page).toHaveURL(/\/conta$/);
```

As assertões web aguardam e reavaliam a condição até ela passar ou atingir o timeout.

Dica: use `await`. Sem ele, o teste pode terminar antes da verificação.



Acesse:

- documentação: playwright.dev;
- instalação: playwright.dev/docs/intro;
- repositório:
github.com/microsoft/playwright;
- extensão: procure **Playwright Test for VS Code** no Marketplace.

Para o fluxo TypeScript, instale antes:

- Node.js em versão oficialmente suportada;
- npm, yarn ou pnpm;
- um editor, preferencialmente VS Code;
- Git, recomendado para versionar testes e configuração.



```
node --version  
npm --version  
git --version
```

Na documentação consultada em agosto de 2026, as linhas suportadas do Node.js são 22.x, 24.x e 26.x.

Prática segura: consulte novamente os requisitos oficiais antes de instalar em laboratório ou CI; versões suportadas mudam com o tempo.



```
npm init playwright@latest
```

O assistente pergunta:

- TypeScript ou JavaScript;
- nome da pasta de testes;
- criação de workflow para GitHub Actions;
- instalação dos navegadores.

Para esta aula, escolha **TypeScript**, pasta tests e instalação dos navegadores.



```
projeto/  
|-- playwright.config.ts  
|-- package.json  
|-- package-lock.json  
`-- tests/  
    |-- example.spec.ts
```

- `playwright.config.ts`: navegadores, timeouts, retries e relatórios;
- `tests/`: especificações automatizadas;
- `package.json`: dependências e scripts;
- cache do sistema: binários dos navegadores.



```
npx playwright install
```

Instala os navegadores esperados pela versão do Playwright.

Em Linux ou CI:

```
npx playwright install --with-deps
```

Para somente Chromium:

```
npx playwright install chromium
```

Ao atualizar o Playwright, pode ser necessário instalar novamente os binários.



```
import { test, expect } from '@playwright/test';

test('permite login com credenciais válidas', async ({ page }) => {
  await page.goto('http://localhost:3000/login');

  await page.getByLabel('E-mail').fill('ana@exemplo.com');
  await page.getByLabel('Senha').fill('SenhaSegura123!');
  await page.getByRole('button', { name: 'Entrar' }).click();

  await expect(page).toHaveURL(/\/conta$/);
  await expect(page.getByRole('heading', {
    name: 'Minha conta'
  })).toBeVisible();
});
```



- Parte — Código — Função
- preparação — `page.goto(...)` — abrir o estado inicial
- entrada — `fill(...)` — informar os dados
- ação — `click()` — disparar o comportamento
- oráculo — `expect(...)` — comparar observado e esperado

Observe: o teste ainda precisa controlar a existência e o estado da conta de Ana.



```
test('nega login com senha inválida', async ({ page }) => {  
  await page.goto('/login');  
  
  await page.getByLabel('E-mail').fill('ana@exemplo.com');  
  await page.getByLabel('Senha').fill('senha-incorreta');  
  await page.getByRole('button', { name: 'Entrar' }).click();  
  
  await expect(page.getByRole('alert'))  
    .toHaveText('E-mail ou senha inválidos');  
  await expect(page).toHaveURL(/\/login$/);  
});
```

Verifique também que sessão e conteúdo protegido não foram criados.



```
import { defineConfig, devices } from '@playwright/test';

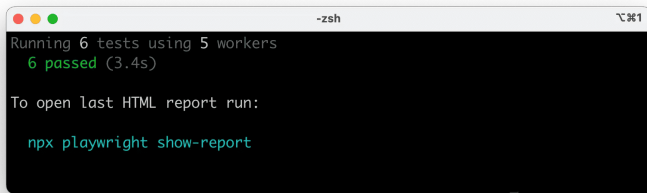
export default defineConfig({
  testDir: './tests',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

Projetos executam a mesma intenção em configurações diferentes.

Execute a suíte pelo terminal



```
npx playwright test
```



```
-zsh 1
Running 6 tests using 5 workers
  6 passed (3.4s)

To open last HTML report run:

  npx playwright show-report
```

O terminal apresenta quantidade de testes, workers, resultados e duração.
Imagem: documentação oficial do Playwright.



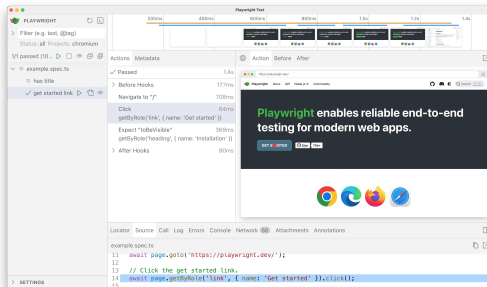
```
npx playwright test --headed  
npx playwright test --project=chromium  
npx playwright test tests/login.spec.ts  
npx playwright test --debug
```

Use `--headed` para observar o navegador e `--debug` para pausar e inspecionar a execução.

UI Mode aproxima código, ações e página



```
npx playwright test --ui
```



Use para executar testes individualmente, acompanhar ações, explorar snapshots do DOM, rede, console e código-fonte.

Imagem: documentação oficial do Playwright.



```
npx playwright codegen http://localhost:3000
```

O gerador registra interações e sugere código.

Use-o para:

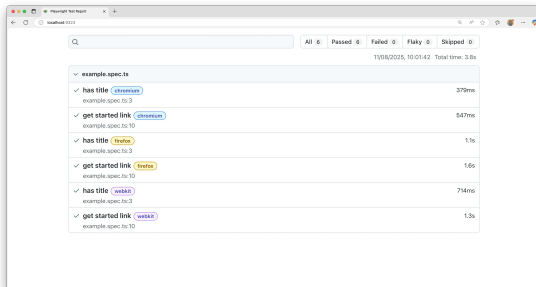
- explorar uma aplicação desconhecida;
- descobrir locators;
- criar um rascunho de fluxo.

Não pare no código gerado: renomeie o teste, remova passos acidentais, prepare dados e acrescente oráculos relevantes.

O relatório transforma execução em evidência



```
npx playwright show-report
```



O relatório permite filtrar testes aprovados, reprovados, ignorados e instáveis, além de abrir erros, passos e anexos.

Imagem: documentação oficial do Playwright.



Configure:

```
use: {  
  trace: 'on-first-retry',  
}
```

Ou force localmente:

```
npx playwright test --trace on
```

O trace registra ações, snapshots, rede, console e fontes para investigar o estado antes e depois de cada passo.



```
await page.screenshot({  
  path: 'evidencias/login.png',  
  fullPage: true,  
});
```

Uma captura demonstra aparência, mas não prova sozinha:

- persistência correta;
- autorização;
- ausência de duplicidade;
- integridade do banco;
- emissão correta de eventos.

Colete a evidência que sustenta a conclusão do caso de teste.



Opções comuns:

- criar dados por API antes do teste;
- usar fixtures;
- carregar um estado conhecido;
- gerar identificadores únicos;
- limpar dados ao final;
- restaurar banco ou ambiente descartável.

Evite: depender de uma conta compartilhada que pode estar bloqueada, alterada ou sendo usada por outra execução.



O Playwright pode executar testes em workers paralelos.

Para isso funcionar:

- cada teste cria ou recebe seus próprios dados;
- nenhum teste depende da ordem;
- portas, arquivos e contas não entram em conflito;
- o ambiente suporta acessos simultâneos;
- o teste não reutiliza estado mutável sem controle.

Comece com independência; paralelismo será consequência.



Priorize fluxos:

- críticos para receita, operação ou segurança;
- repetidos com frequência;
- estáveis o suficiente para manter;
- com resultado esperado claro;
- executados em navegadores importantes;
- caros ou demorados quando manuais.

Não automatize uma regra ambígua apenas para produzir mais testes.



- usar `waitForTimeout` como solução padrão;
- selecionar por CSS acoplado ao layout;
- compartilhar usuário e dados entre testes;
- verificar somente que "algo apareceu";
- esconder falhas com retries excessivos;
- depender de serviço externo sem controle;
- ignorar diferenças entre navegadores;
- misturar muitas regras em uma única jornada longa.

Teste instável não é inevitável: normalmente existe estado, sincronização ou seletor mal controlado.



Regra: idade deve ser inteira de 18 a 65.

```
const casos = [  
  { idade: '17', aceito: false },  
  { idade: '18', aceito: true },  
  { idade: '65', aceito: true },  
  { idade: '66', aceito: false },  
  { idade: 'dezoito', aceito: false },  
];  
  
for (const caso of casos) {  
  test(`idade ${caso.idade}`, async ({ page }) => {  
    // preparar, preencher, enviar e verificar  
  });  
}
```

A parametrização executa uma decisão de projeto já tomada.



Em duplas:

- 1 instale o Playwright em um projeto;
- 2 abra uma aplicação local ou ambiente autorizado;
- 3 crie `tests/login.spec.ts`;
- 4 implemente um cenário válido e um inválido;
- 5 use locators por papel ou rótulo;
- 6 verifique URL, mensagem e estado autenticado;
- 7 execute em Chromium com `--headed`;
- 8 provoque uma falha e investigue no UI Mode.

Registre quais precondições e dados foram necessários.



- requisito e resultado esperado definidos;
- ambiente autorizado e endereço configurado;
- dados controlados;
- teste independente;
- locators semânticos e únicos;
- ações aguardadas com `await`;
- asserções web específicas;
- efeitos importantes verificados;
- execução em navegador relevante;
- relatório ou trace disponível para falhas;
- segredos fora do código e do repositório.



- Playwright automatiza a execução do caso funcional no navegador.
- Browser, context, page e locator separam controle e isolamento.
- Locators semânticos e auto-wait reduzem fragilidade.
- Asserções transformam o resultado esperado em verificação.
- Projetos repetem a intenção em navegadores e dispositivos.
- UI Mode, relatório e trace tornam falhas investigáveis.
- Dados, condições e oráculos continuam sendo responsabilidade do projeto de teste.



- Instalação — playwright.dev/docs/intro
- Escrita de testes — playwright.dev/docs/writing-tests
- Locators — playwright.dev/docs/locators
- Asserções — playwright.dev/docs/test-assertions
- Auto-waiting — playwright.dev/docs/actionability
- Navegadores — playwright.dev/docs/browsers
- UI Mode — playwright.dev/docs/test-ui-mode
- Trace Viewer — playwright.dev/docs/trace-viewer-intro
- Test generator — playwright.dev/docs/codegen

Documentação consultada em 16 de agosto de 2026.